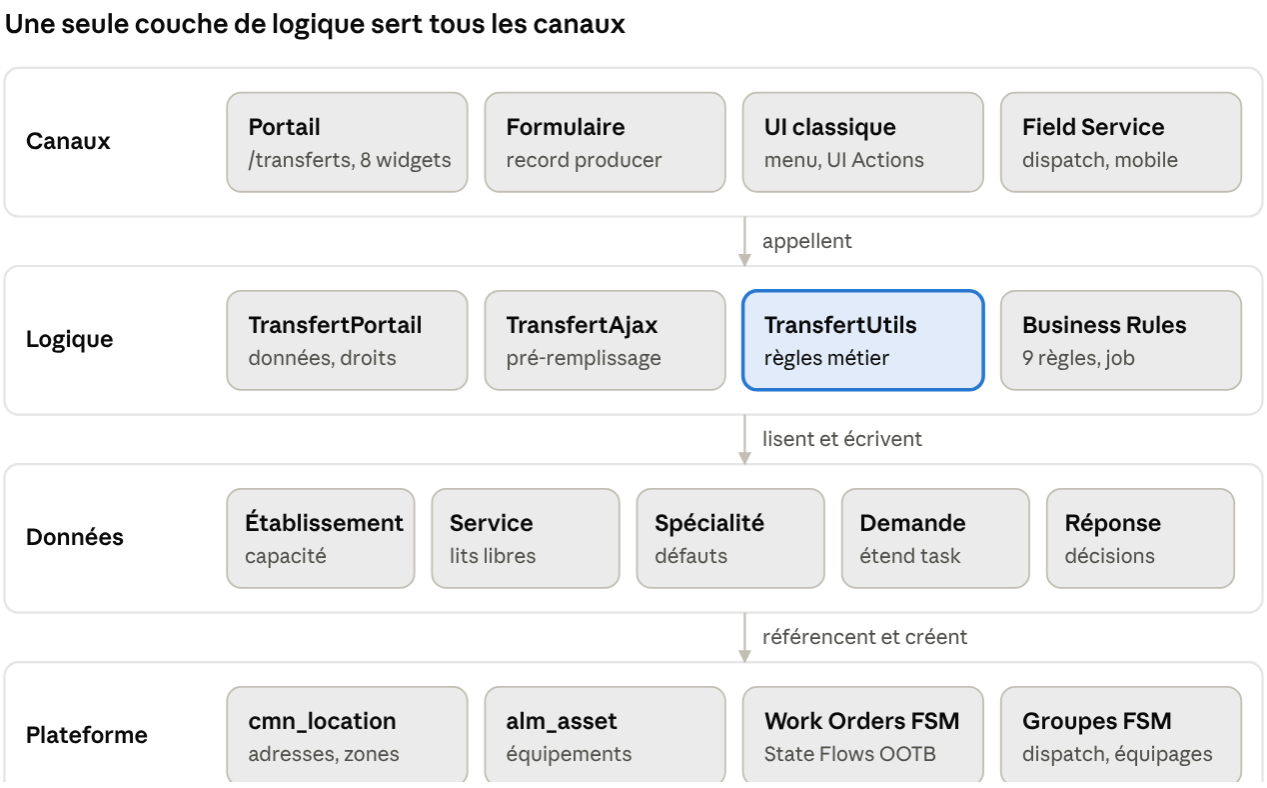
Transfert Patients — Documentation d'architecture

Oct 15, 2026 · @Someone

Vue d'ensemble

Transfert Patients est une application scopée ServiceNow : quatre canaux s'appuient sur une couche de logique unique, centrée sur `TransfertUtils`, qui écrit dans huit tables métier reliées aux tables OOTB et à Field Service.



architecture en couches · 4 niveaux

Le portail et le formulaire n'embarquent aucune règle métier : ils appellent `TransfertPortail` et `TransfertAjax`, qui délèguent à `TransfertUtils`.

Principes d'architecture

1. **Une application scopée** (`x_2218203_ptrans`) isole tout le code et les données métier ; rien n'est modifié dans le scope global.
2. **Réutiliser l'OOTB** : localisations `cmn_location`, équipements `alm_asset` / `cmdb_model`, groupes, tâches `task`, Field Service (`wm_order`, `wm_task`), Service Catalog, Service Portal, notifications.

3. **Field Service reste intact** : l'application crée des ordres standard et les fait avancer uniquement par les State Flows OOTB, selon la configuration `sm_config` de l'instance.
4. **Une seule logique métier** : les règles sont dans des Script Includes appelés par les Business Rules, les UI Actions, le formulaire et le portail ; aucun canal ne réimplémente une règle.
5. **Droits vérifiés côté serveur** pour chaque action du portail, en plus des ACL.
6. **Données patient minimales** : référence pseudonymisée et besoin clinique uniquement.
7. **Infrastructure as code** : toute la configuration est déployée par des scripts idempotents versionnés dans Git.

Composants applicatifs

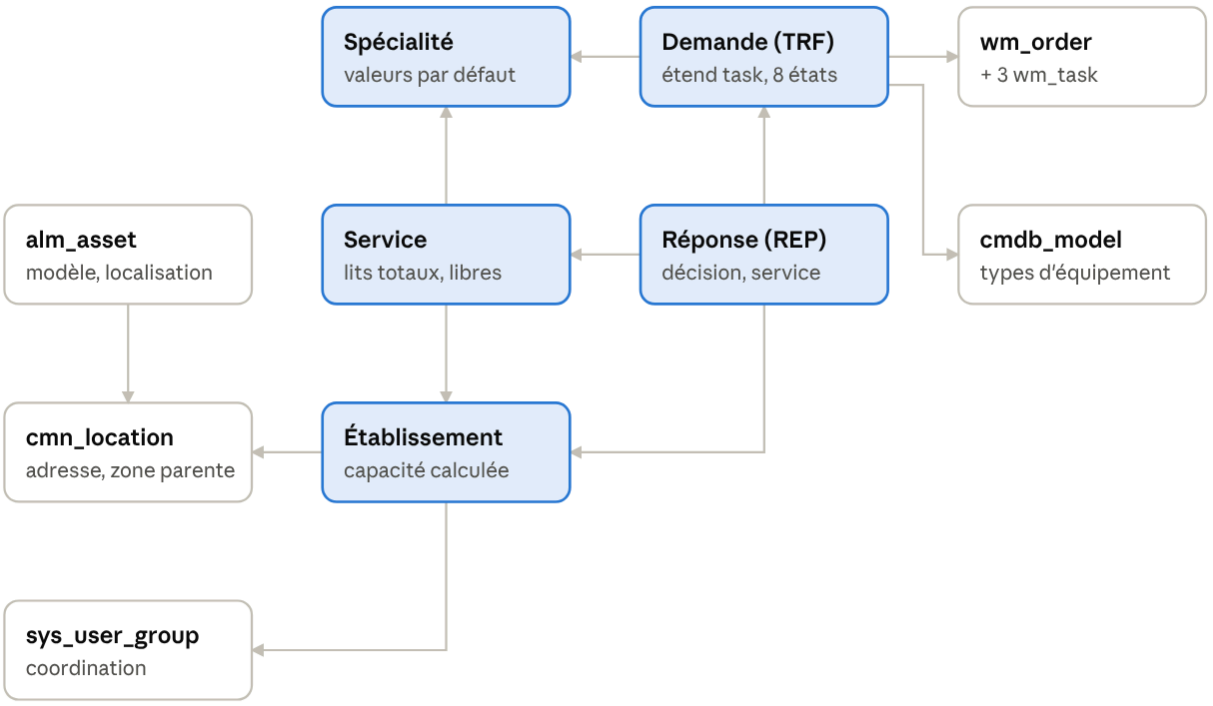
Composant	Type	Responsabilité
`TransfertUtils`	Script Include	Règles métier : éligibilité et publication, réservation du lit, réservation et libération des équipements et du véhicule, création de l'ordre FSM et transitions OOTB, synchronisation FSM, capacité, expiration, republication, rattachement d'un utilisateur à ses établissements
`TransfertPortail`	Script Include	Données des widgets (accueil, listes, détail, régulation) et actions du portail (dont republier et relancer) avec contrôle des droits
`TransfertAjax`	Script Include client-callable	GlideAjax du formulaire : contexte du demandeur, suggestions par spécialité
10 Business Rules	Business Rules	Validation, publication, clôture des réponses, contrôle des décisions, compteurs, retenue, libération des réservations, capacité, suivi de `wm_order`
6 UI Actions	UI Actions	Publier, Annuler, Republier, Accepter, Refuser, Retenir (interface classique)
Job planifié	Scheduled Script	Expiration des demandes toutes les minutes

Composant	Type	Responsabilité
Record producer	Service Catalog	Formulaire guidé de création, variables mappées, 3 client scripts, publication post-insertion
Portail `/transferts`	Service Portal	Portail « Waluu » : thème, en-tête, 9 widgets temps réel, 5 pages dont la carte des hôpitaux (Leaflet + OpenStreetMap)
6 notifications	Email	Nouvelle demande, réponse, destination confirmée, non retenue, expiration, relance
Fix Script ACL	Fix Script	Création des ACL (exécuté en `security_admin`)
Groupes et zones	Données FSM	3 zones, 12 véhicules sanitaires (alm_asset), groupes `wm_dispatch`, `wm_work`, `wm_qualification`
4 définitions de SLA	Service Level Management (OOTB)	Délais de réponse (5 / 15 min) et de prise en charge (10 / 15 min) des demandes vitales et urgentes ; source de vérité des délais, lue par `TransfertUtils.delaiMin()` ; sans flux de notification

Modèle de données

Huit tables portent le métier ; elles référencent les tables OOTB plutôt que de les dupliquer. Le schéma montre le cœur du modèle ; les tables Relance, Réservation d'asset et Règle d'équipement figurent dans le tableau des relations.

Demande et Réponse relient le référentiel à Field Service



Flèche : champ référence vers la table cible · teinté : tables de l'application · neutre : tables OOTB

modèle de données · 5 tables de l'application, 5 tables OOTB

Relation	Champ
Demande → Établissement	`etablissement_origine`, `etablissement_destination`
Demande → Service	`service_origine`, `service_destination`
Demande → Spécialité	`specialite_requise`
Demande → cmdb_model	`equipements_requis` (liste)
Demande → wm_order	`work_order`
Demande → Réponse	`reponse_retenue`
Réponse → Demande, Établissement, Service	`demande`, `etablissement`, `service_propose`
Service → Établissement, Spécialité	`etablissement`, `specialite`
Établissement → cmn_location, sys_user, sys_user_group	`location`, `coordinateur`, `groupe_coordination`
Spécialité → cmdb_model	`equipements_defaut` (liste)

Relation	Champ
alm_asset → cmn_location, cmdb_model	`location`, `model` (OOTB)
Demande → alm_asset (véhicule)	`vehicule`
Demande → Demande (republication)	`demande_precedente`, `republication`
Relance → Demande, Établissement, Service	`demande`, `etablissement`, `service_propose`
Réservation d'asset → alm_asset, Demande, Établissement	`asset`, `demande`, `etablissement` (vide pour un véhicule)
Règle d'équipement → cmdb_model	`modele` (+ `quantite_minimum`)

Intégration Field Service

L'application crée l'ordre et ses tâches et assigne les tâches ; la suite (qualification, acceptation, exécution, clôture) est le circuit Field Service standard. Le Work Order n'est jamais modifié directement : son état suit celui des tâches par les règles OOTB.

Point d'intégration	Mécanisme
Création	`wm_order` (localisation = origine, priorité = urgence) + 1 `wm_task` « Transport du patient », une seule tâche située à l'origine (étapes OOTB Start Travel, Check-In, Start Work, Close Complete) ; avant le 16/10/2026 : 3 tâches
Première transition	`global.StateFlow` : le script essaie dans l'ordre les State Flows OOTB *Ready for Dispatch*, *Ready For Qualification*, *Ready For Approval* et applique le premier que `validFlow` autorise. Sur l'instance (`sm_config` : qualification active), l'ordre passe en *Awaiting Qualification*.
Groupes	Les règles OOTB *Populate Group - Dispatch/Work* renseignent le groupe de qualification et le groupe de dispatch d'après `sys_group_covers_location`, en remontant la hiérarchie `cmn_location`
Zones	3 localisations parentes (Dakar Plateau-Médina, Fann-Grand Yoff, Pikine-Guédiawaye) sous « Région de Dakar » ; chaque zone a un groupe `wm_dispatch` et un groupe `wm_work`

Point d'intégration	Mécanisme
Retour	Business Rule after update sur `wm_order` (scope de l'application) : *Work In Progress* → En transit, *Closed Complete* → Arrivé, *Cancelled* | *Closed Incomplete* → note d'alerte
Véhicule	Réservé avant la création de l'ordre ; cité dans la description du `wm_order` et de la tâche de transport, et porté par le champ `vehicule` de la demande : `wm_order` et `wm_task` n'ont pas de champ asset, FSM n'est pas modifié
Assignation des tâches	`assignerTaches()` renseigne `assignment_group`, `assigned_to`, `expected_start` et les durées ; le state flow OOTB « Assigned » fait passer la tâche en Assigned. Les contrôles OOTB s'appliquent : groupe autorisé (Populate Group - Dispatch/Work) et conflit de planning de l'agent (Date Checks)
Dépendances de groupes	Avec la file de dispatch active, FSM n'accepte un équipage sur une tâche que s'il couvre sa localisation et dépend de son groupe de dispatch (`sm_m2m_group_dependency`). Les 3 équipages couvrent toute la région et dépendent des 3 groupes de dispatch (9 liens)
Suivi dans le portail	Lecture seule des champs OOTB de `wm_task` (prévu : expected_start, estimated_work_duration, estimated_end ; réel : actual_travel_start, work_start, work_end), de l'historique `sys_audit` des états (heures d'assignation et d'acceptation) et du journal work_notes ; rafraîchi par recordWatch quand l'ambulancier agit sur mobile
Itinéraire routier	À la retenue, appel sortant `sn_ws.RESTMessageV2` vers OSRM (/route/v1/driving, tracé encodé polyline) : tracé, distance et durée × coefficient de circulation stockés sur la demande ; la durée alimente la tâche Transport. Un seul appel par demande, délai d'attente 8 s, repli sur l'estimation à vol d'oiseau

La tâche unique est située à l'origine : un seul groupe de dispatch (zone d'origine), même pour un transfert inter-zones : c'est pour cela que les équipages couvrent toute la région et dépendent des trois groupes de dispatch.

Sécurité

La sécurité repose sur deux niveaux : les ACL sur les tables et un contrôle des droits dans chaque action serveur du portail.

Rôle	Attribué à	Droits principaux
`x_2218203_ptrans.admin`	administrateurs fonctionnels	Référentiel complet
`x_2218203_ptrans.regulateur`	régulateurs	Toutes les demandes et réponses, retenir, annuler
`x_2218203_ptrans.coordinateur`	groupes « Coordination - … »	Lecture des demandes publiées à son établissement, réponse, capacité de ses services
`x_2218203_ptrans.demandeur`	agents, groupes de coordination	Création, lecture et annulation de ses demandes
`wm_qualifier`, `wm_dispatcher`, `wm_agent`	groupes FSM	Rôles Field Service standard
`x_2218203_ptrans.ambulancier`	groupes « Équipages ambulance - … »	Page « Mes missions » du portail (et l'accueil, redirigé) ; actions sur ses seules tâches

- **ACL** : livrées dans le Fix Script « Transfert Patients - créer les ACL », en scope global (un script scopé ne peut pas créer d'ACL ; il les rattache à l'application), car leur création exige aussi l'élévation `security_admin`. Les ACL à condition (« son établissement ») appellent `TransfertUtils.estCoordinateur()`.
- **Contrôles serveur** : `TransfertPortail` vérifie le rôle ou le rattachement avant chaque action (répondre, ajuster les lits, retenir, publier, annuler, republier, relancer) et avant d'afficher une demande.
- **Intégrité** : les Business Rules rejettent une décision incohérente (service d'un autre établissement, plus de lit, motif absent, retenue hors état Publiée).
- **Données patient** : référence pseudonymisée, âge, sexe, besoin clinique ; aucun nom ni identifiant national.

Flux principaux

Publication

1. Le formulaire (ou le bouton Publier) passe la demande à l'état Publiée.
2. La Business Rule before « Demande - publication » appelle `TransfertUtils.publier()`.
3. `publier()` cherche les services éligibles, vérifie que chaque établissement a, pour chaque équipement requis, le minimum d'unités libres (règle d'équipement, réservations actives déduites), crée une réponse « En attente » par établissement et fixe la date limite.
4. L'insertion des réponses déclenche la notification « Nouvelle demande à traiter ».

Réponse et retenue

1. Le widget appelle `TransfertPortail.repondre()`, qui vérifie le rattachement puis met à jour la réponse.
2. La Business Rule « Réponse - contrôle décision » valide et horodate ; « compteur acceptations » met à jour la demande.
3. Le régulateur appelle `retenir()` : la décision passe à Retenue, la Business Rule after appelle `TransfertUtils.retenir()`, qui réserve le lit et les équipements, clôture les autres réponses et appelle `creerOrdreTransport()`.

Réservation et libération des assets

1. `TransfertUtils.retenir()` appelle `reserverEquipements()` : une unité libre de chaque modèle requis dans l'établissement retenu.
2. `creerOrdreTransport()` appelle `affecterVehicule()` : un véhicule libre d'un modèle compatible avec le mode de transport, cherché dans la localisation de l'origine puis ses parents (zone, région), puis partout.
3. Chaque réservation est un enregistrement `x_2218203_ptrans_reservation` au statut Active ; un asset qui a une réservation active n'est plus compté libre.
4. À l'état Arrivé, Annulée ou Expirée, la Business Rule « Demande - libération des réservations » appelle `libererReservations()` : statut Libérée, date et motif.

Expiration, republication et relance

1. Le job planifié appelle `expirerDemandes()` : demande Expirée, réponses en attente Expirées.
2. `TransfertUtils.republier()` copie la demande, la relie à l'ancienne (`demande_precedente` / `republication`), la publie et passe les relances en Traitée.
3. `TransfertPortail.relancer()` crée une relance (RLC) ; la notification part à l'auteur et à la coordination d'origine.

Temps réel

Chaque widget s'abonne aux tables qui le concernent avec `spUtil.recordWatch` (demandes, réponses, services, `wm_task`) et relance son script serveur à chaque modification ; aucun rafraîchissement manuel.

Les comptes à rebours (cartes à traiter, page d'une demande, tableau de régulation) tournent dans le navigateur chaque seconde (`$interval`) à partir de l'échéance et de l'heure du serveur (`limiteMs`, `maintenantMs`) : l'écart d'horloge du poste est corrigé et aucun appel serveur n'est fait.

Les minuteurs d'étape (réponse, décision, arrivée, bilan) sont calculés côté serveur par `TransfertPortail._minuteur()` (phase, échéance, début, bilan) puis animés chaque seconde dans le navigateur, sur la page d'une demande, le tableau de régulation, les cartes à traiter et les arrivées attendues.

Déploiement et environnements

Élément	Valeur
Instance	PDI `dev408543.service-now.com`, release Australia
Application	Transfert Patients, scope `x_2218203_ptrans`
Plugin requis	Field Service Management (`com.snc.work_management`)
Plugins utilisés déjà actifs	Asset Management, Service Catalog, Service Portal, Notifications, Flow Designer
Source	Repo Git, dossier `TransfertPatients/`
Déploiement	9 scripts Python idempotents (`build/01` à `09`) via l'API REST Table, avec la préférence `apps.current_app` pour écrire dans le scope
Secrets	`.env.servicenow`, ignoré par Git

Pour passer vers une autre instance : publier l'application dans un Update Set ou depuis Studio, ou relancer les scripts après avoir adapté l'identifiant de l'application et les identifiants des State Flows si la release diffère.

Comptes techniques

Compte	Rôles	Usage
`diapandal`	`admin`, `snc_basic_auth_api_access`	Compte d'intégration actif : déploiement par l'API REST (`.env.servicenow`), exécution du job d'expiration (`run_as`). Préférences `apps.current_app` = Transfert Patients et update sets de l'application.
`adminclaude`	`admin`, `snc_basic_auth_api_access`	Ancien compte d'intégration, conservé actif ; auteur de l'historique de construction et des scénarios de démo.

Les notes écrites par les Business Rules sont attribuées à l'utilisateur qui déclenche l'action : sous le compte d'intégration quand l'action passe par l'API, sous l'utilisateur métier quand elle passe par le portail.

Technologies et outils

Waluu est construit à 100 % sur ServiceNow (Field Service, gestion des actifs, SLA, Service Portal), complété par des briques cartographiques open source sans clé d'API.

Famille	Technologies et outils	Rôle
Plateforme	ServiceNow release Australia (PDI), application scopée `x_2218203_ptrans`	Socle de l'application, 8 tables métier
Logique serveur	Script Includes, Business Rules, UI Actions, job planifié, propriétés système	Règles métier, publication, réservations, expiration
Field Service Management	`wm_order`, `wm_task`, state flows OOTB, groupes de dispatch, d'équipages et de qualification	Organisation et suivi du transport
Gestion des actifs	`alm_asset`, `cmdb_model`, `cmn_location`	Équipements médicaux, véhicules, positions GPS
Délais	Moteur de SLA OOTB (4 définitions)	Délais de réponse et de prise en charge

Famille	Technologies et outils	Rôle
Saisie	Service Catalog (record producer, scripts client, GlideAjax)	Formulaire guidé de demande
Portail	Service Portal (10 widgets, 7 pages, Record Watcher)	Interface temps réel de tous les acteurs
Notifications	Moteur de notifications email (6 notifications)	Alertes aux coordinateurs et demandeurs
Sécurité	Rôles de l'application, ACL (Fix Script)	Droits par acteur
Intégration	`RESTMessageV2` (appel sortant)	Itinéraire routier
Cartographie	Leaflet 1.9.4 (cdnjs), fonds OpenStreetMap	Cartes interactives du portail
Itinéraires	OSRM (serveur public de démonstration)	Tracé, distance et durée de conduite
Langages	JavaScript (serveur ES5, AngularJS), HTML, SCSS, Python 3	Code de l'application et des scripts de build
Construction et documentation	API REST ServiceNow, client `tools/sn.py`, scripts `build/01` à `10`, Git, python-pptx, Claude Docs	Déploiement reproductible, présentations, documentations

Mesure des temps de trajet

Trois mécanismes se succèdent selon le moment du transfert.

Moment	Moteur	Calcul	Usage
Publication	Calcul interne	Haversine sur les coordonnées GPS × 1,3, à 25 km/h, 5 min minimum	Tri « le plus proche », indication « tient le délai » ; repli si OSRM ne répond pas
Retenue	OSRM sur les routes OpenStreetMap	Durée de conduite sans circulation × coefficient 1,5 (propriété)	Durée de la tâche Transport, arrivée estimée, progression sur la carte

Moment	Moteur	Calcul	Usage
Pendant et après le trajet	Field Service	Horodatages réels : départ, sur place, prise en charge, remise	SLA, bilan des délais

Le calcul de trajet intégré à Field Service (Google Maps) n'est pas utilisé : il exige une clé d'API absente du PDI.

Envoi des demandes aux établissements

Aucun moteur externe : la Business Rule « Demande - publication » appelle `TransfertUtils.publier()`, qui crée une réponse « En attente » par établissement retenu. La diffusion passe par deux canaux : le portail, qui affiche la demande instantanément grâce au Record Watcher, et la notification email créée à l'insertion de la réponse. Sur le PDI, l'envoi des emails est désactivé (`glide.email.smtp.active = false`) : seuls le portail et les notifications en boîte d'envoi fonctionnent.

Décisions d'architecture

Décision	Raison	Alternative écartée
La demande étend `task`, le transport est un `wm_order` séparé	FSM n'a pas de mécanisme de publication ; FSM reste OOTB et évolutif	Publier directement des Work Orders
Établissement = table propre liée à `cmn_location`	`cmn_location` n'est pas extensible	Étendre `cmn_location`
Équipements = assets OOTB localisés	Réutilise Asset Management, sans table spécifique	Table d'équipements custom
Transitions FSM par `StateFlow` avec `validFlow`	Respecte la configuration de chaque instance (qualification, approbation)	Forcer l'état du Work Order
Le régulateur choisit parmi les acceptations	Décision médicale et logistique humaine	Première acceptation retenue automatiquement
Publication aux seuls établissements éligibles	Moins de sollicitations inutiles	Publication à tous
Back-office en pages de portail	Un workspace UI Builder n'est pas créable de façon fiable par l'API REST	Workspace UI Builder

Décision	Raison	Alternative écartée
ACL dans un Fix Script	L'API ne peut pas obtenir l'élévation `security_admin`	Création par l'API

Décisions prises après la démonstration du 14 octobre 2026 et le 15 octobre 2026 :

Décision	Raison
Le Fix Script ACL vit en scope global et rattache chaque ACL à l'application	Un script scopé ne peut pas insérer dans `sys_security_acl` ; les insertions échouaient sans erreur
L'ordre de transport est relié à la demande dès sa création ; tâches et transitions FSM sont isolées	Pour un utilisateur sans rôle FSM, une règle Field Service lève une exception après la création des tâches
Le déclencheur des scripts onChange du formulaire est écrit côté serveur	Le champ `cat_variable` est une liste de choix dynamique que l'API Table refuse
Après envoi depuis le portail, redirection vers le parcours (`producer.portal_redirect`)	Le portail ouvrait la page ticket standard
Les réservations d'assets vivent dans une table de l'application ; le statut de l'`alm_asset` n'est pas modifié	Le statut appartient à la gestion du parc ; un appareil réservé reste en service. La table trace qui, quand, pourquoi, et la libération est automatique
Quantité minimum par modèle dans une table de règles	Paramétrable par l'administrateur sans code ; aucun champ ajouté à `cmdb_model`, hors du scope
Véhicules = assets OOTB localisés sur la zone	Même mécanisme que les équipements ; la zone est déjà le parent des localisations d'établissement, ce qui donne la recherche de proximité
Compte à rebours calculé dans le navigateur	Affichage à la seconde sans charge serveur ; le passage en Expirée reste fait par le job
Une seule tâche « Transport du patient » (16/10/2026), assignée dès la retenue	Un même équipage prend le patient en charge de bout en bout ; FSM refuse deux tâches simultanées pour un agent et compte 1 h de trajet par défaut : trajets fixés à 30 min, 0, 0. Le Work Order n'est pas modifié

Décision	Raison
Suivi des tâches calculé à partir des données FSM existantes, sans champ ni table ajoutés	Les horaires réels et les changements d'état sont déjà saisis par l'ambulancier dans Field Service (mobile) ; le portail n'en est qu'une vue
Utilisateurs de démo au fuseau GMT	Le PDI n'a pas de fuseau par défaut (Los Angeles) : les heures du portail étaient décalées de 7 h pour Dakar
Distance calculée une fois à la publication et stockée sur la réponse (haversine sur les coordonnées de `cmn_location`)	Aucune dépendance externe (pas de clé d'API cartographique sur le PDI) ; tri et affichage sans recalcul ; colonne disponible dans les listes et rapports
Délais portés par des SLA OOTB sur la demande (elle étend task)	Modifiables par l'administrateur sans code, échéance et dépassement calculés par le moteur, rapports de respect des délais ; le code relit la durée de la définition pour la date limite, l'arrivée exigée et la fenêtre FSM. Cinq horodatages sur la demande (publication, 1re acceptation, retenue, arrivée exigée, arrivée) alimentent les minuteurs
Fenêtre FSM des tâches = retenue → arrivée exigée ; si intenable, fenêtre étendue à la durée réelle avec alerte	Le planning reste réaliste et accepté par FSM ; le dépassement est porté par le SLA, qui garde l'échéance réglementaire
Carte du portail avec Leaflet (dépendance de widget OOTB) et fonds OpenStreetMap	Aucune clé d'API (la carte FSM exige Google Maps) ; positions lues dans cmn_location ; en production, utiliser un fournisseur de tuiles (la politique d'usage des serveurs OpenStreetMap ne couvre pas un usage intensif)
Itinéraire OSRM calculé seulement à la retenue, durée × 1,5 (option A)	OSRM donne une durée sans circulation ; le coefficient la rend réaliste pour Dakar. Un seul appel par demande. Le serveur public OSRM sert à la démonstration ; en production, OSRM hébergé sur les données du Sénégal ou fournisseur payant (propriété x_2218203_ptrans.itineraire.osrm_url)

Décision	Raison
Page portail « Mes missions » en attendant l'application mobile Field Service	La cible est l'application Agent (Field Service Mobile : GPS réel, notifications, hors connexion), bloquée par le téléchargement depuis le Store sur le PDI. La page applique les mêmes state flows OOTB (global.StateFlow, exécutés sous l'utilisateur connecté) : elle reste cohérente avec l'app mobile et pourra être retirée
Passage de 3 tâches à une tâche unique (16/10/2026)	Le même ambulancier faisait les 3 tâches d'un trait : planning enchaîné forcé, 3 acceptations, 6 actions, 2 dispatchers en inter-zones, et une préparation qui relève du personnel de l'origine. Les étapes OOTB d'une tâche (départ, sur place, début, fin) donnent le même suivi ; une tâche peut être ajoutée si une étape est confiée à une autre équipe

Limites et évolutions

Limite	Évolution proposée
Lit non libéré si la demande ou le transport est annulé après la retenue	Étendre aux lits la table « Réservation d'asset » et sa libération automatique
Pas de blocage du lit à l'acceptation	Blocage temporaire avec expiration
Remise inter-zones routée vers la zone de destination	Fusionner transport et remise en une tâche
Tâche créée et assignée par script, sans compétences ; durée de remise fixe (2 min)	Work Order Templates, dépendances `sm_m2m_task_dependency`, compétences `cmn_skill`
Back-office dans le portail	Workspace UI Builder réutilisant `TransfertPortail`
Distance à vol d'oiseau et durée de route estimée à vitesse fixe ; la distance ne pondère pas la sollicitation	Temps de trajet réel par l'API cartographique de Field Service (SMGeoDistanceUtils, clé Google Maps) et rayon maximal par urgence, à partir des coordonnées de `cmn_location`

Limite	Évolution proposée
Le véhicule n'est pas une ressource planifiable dans FSM (cité en texte)	CI véhicule et affectation de ressources FSM, ou champ asset sur la tâche de transport
Une seule unité réservée par équipement requis	Quantité par équipement dans la demande
Dispatcher Workspace FSM non installé : le PDI ne télécharge plus depuis le Store	Installer `sn_fsm_disp_wrkspc` depuis le portail développeur ; en attendant, dispatch dans l'interface classique
Fuseau horaire système non défini (`glide.sys.default.tz` vide) ; seuls les utilisateurs de démo sont en GMT	Fixer le fuseau système à GMT pour tous les utilisateurs de Dakar